Experiment No: 11

Aim: Case Study: CORBA , Android Stack

Theory:

Case Study: Common Object Request Broker Architecture (CORBA)

What is CORBA?

CORBA (Common Object Request Broker Architecture) is a standard defined by the Object Management Group (OMG) that allows communication between objects written in different programming languages and running on different machines.

It is a middleware that manages the communication and data exchange in distributed systems. CORBA's main goal is to enable interoperability between applications regardless of:

- The language they are written in (Java, C++, Python, etc.)
- The operating system they run on (Windows, Linux, etc.)
- The network protocol used.

CORBA Architecture Components

Here’s a breakdown of the CORBA architecture:

1. ORB (Object Request Broker)

- Acts like a communication bridge.
- It allows clients to invoke methods on remote server objects.
- ORB locates the object, sends the request, waits for a response, and returns it to the client.

2. IDL (Interface Definition Language)

- A neutral language to define object interfaces.
- Developers write interfaces in IDL.
- Then tools generate language-specific code (stubs and skeletons) from the IDL.

3. Stub and Skeleton

- Client Stub: Acts as a local representative of the remote object.
- Server Skeleton: Receives the request from ORB and passes it to the actual implementation.
- Generated automatically from IDL.

4. POA (Portable Object Adapter)

- Helps manage the lifecycle and requests for CORBA objects on the server side.

5. IIOP (Internet Inter-ORB Protocol)

- Allows communication between ORBs over the Internet.
- Based on TCP/IP.

6. Naming and Directory Services

- Allow clients to look up and find references to server objects dynamically (like DNS for distributed objects).

Real-World Case Study: Banking System Integration

Background
ABC Bank has multiple departments across various locations. Over the years, different systems were developed:

- Account management system in C++
- Online banking portal in Java
- ATM software in COBOL

All of these need to communicate in real-time without being rewritten.

Problem
The systems were built in different programming languages and ran on different platforms (Windows, Linux, Mainframes). Direct integration would be expensive and risky.

Solution Using CORBA

- CORBA provided a language-neutral and platform-independent integration approach.
- A common IDL was created for operations like:
  - openAccount()
  - checkBalance()
  - transferFunds()
- Using IDL compilers, Java stubs and C++ skeletons were generated.
- ORBs were installed on both client and server machines.
- The front-end (Java-based) banking portal could now invoke methods on the backend (C++ or COBOL) systems using CORBA's ORB.
- Naming Service allowed different departments to find services dynamically.

Result

- Seamless integration across systems.
- Minimal changes to existing code.
- Secure and scalable communication between systems.

Advantages of CORBA

Advantage	Description
Language-Independent	Works across Java, C++, Python, etc.
Platform-Independent	Works across Windows, Linux, etc.
Distributed System Support	Enables object-to-object communication remotely.
Reuse of Legacy Code	Easily integrates older software.
Rich Features	Includes naming, event handling, transaction, and security services.

Disadvantages of CORBA

Disadvantage	Description
Complexity	Setup and configuration are not trivial.
Performance	Slower due to abstraction and middleware.
Hard to Debug	Distributed nature adds complexity.
Less Popular	Modern tech like REST APIs and gRPC replaced it.

Conclusion

CORBA was revolutionary in its time and was heavily used in industries like:

- Banking
- Telecommunications
- Aerospace
- Military

Even though it has been largely replaced by newer technologies, understanding CORBA is crucial for grasping the evolution of distributed systems and middleware architectures.